



Architecture for Galaxy on AnVIL in a VM

August 2026

Outline

An update on the *Galaxy on AnVIL* v2 deployment model

Explorations for brining GalaxyAI to AnVIL

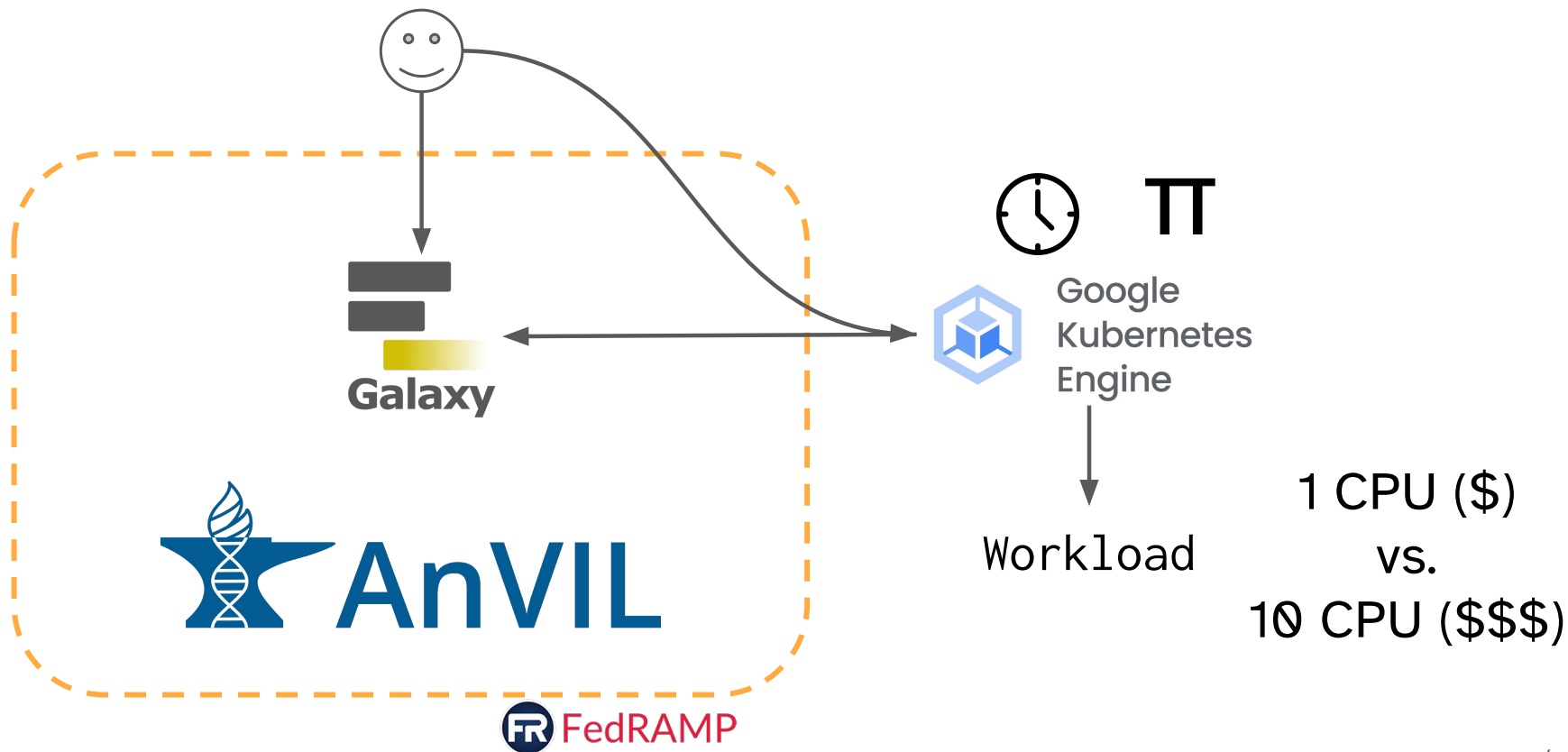
What

Main goal: increase usability of the Galaxy service, and hence usage

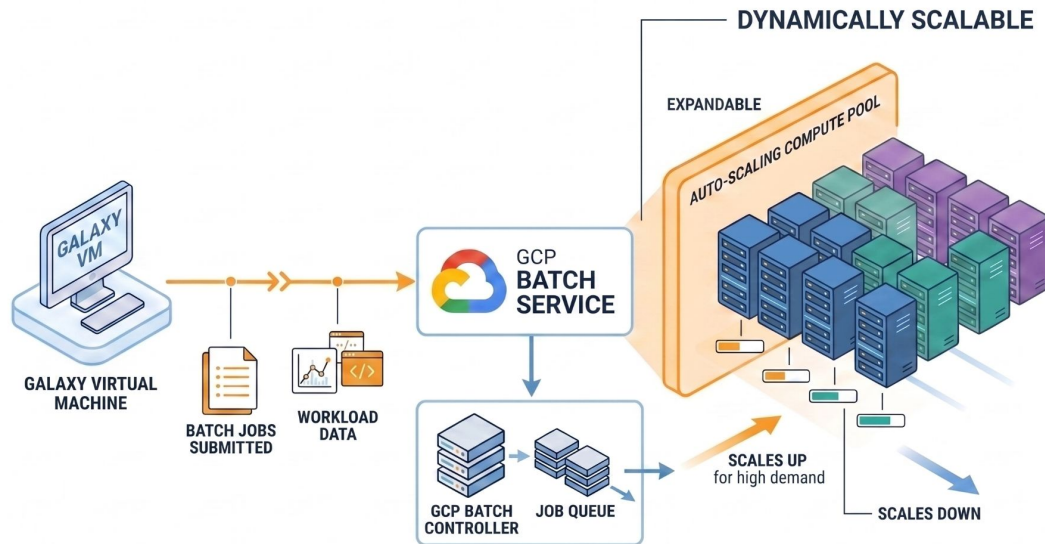
How?

- Quicker startup
- Lower the costs
- Easier updates & bug fixes

Galaxy on AnVIL today



Lightweight Galaxy VM, offloading jobs to GCP Batch for processing



Startup time

Before: ~15 min

Now: about the same 🙄 *

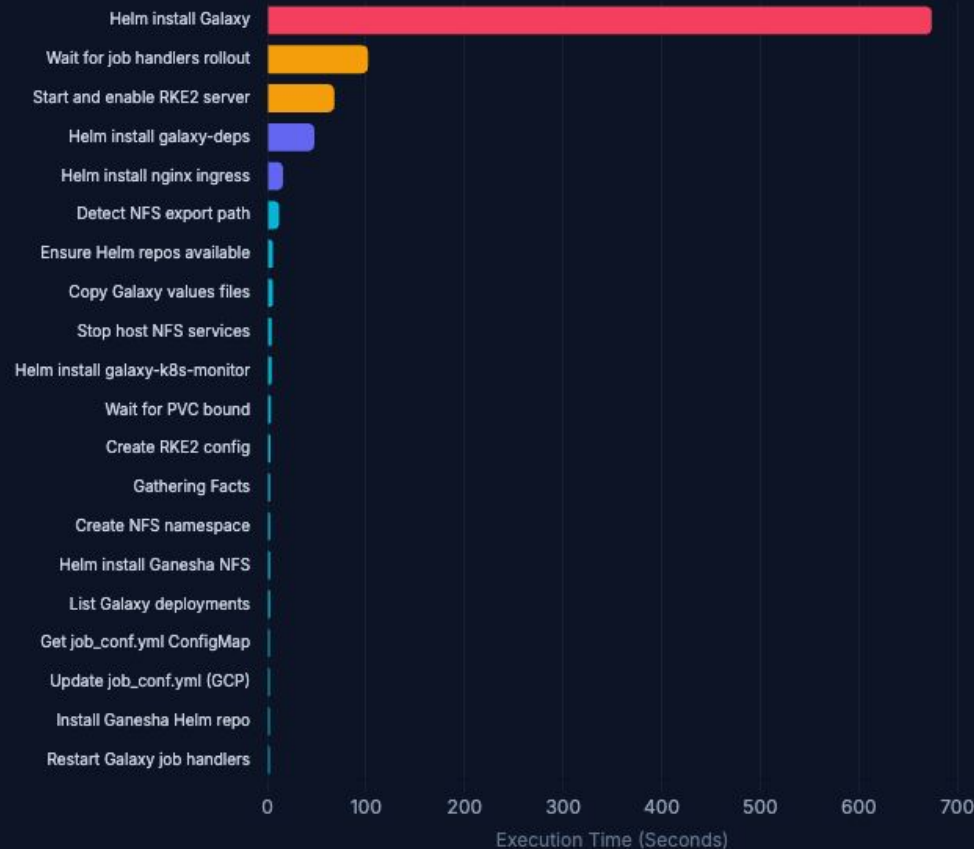
Baseline compute cost

Now: \$0.17/hr (~65%+ lower)

Before: \$0.52+/hr

Ansible Kubernetes Deployment Timings (`galaxy\_k8s\_deployment`)

Total Playbook Duration: 16 minutes 43 seconds (1003.09s across 20 tasks)

Dominant Bottleneck
Helm install Galaxy (67.2%)

TOP DEPLOYMENT TIME SINK

1. Helm Install Galaxy 674.09s (11m 14s)

Consumes 67.2% of total Ansible playbook execution. Represents initial pod startup, schema initialization, and tool loading.

2. Job Handler Rollout 101.97s (1m 42s)
Wait 42s

Takes 10.2% of playbook execution while Kubernetes performs health checks on restarted job handler workers.

3. RKE2 Server Startup 67.68s (1m 07s)

Bootstrapping and enabling single-node RKE2 Kubernetes control plane (6.7% of total time).

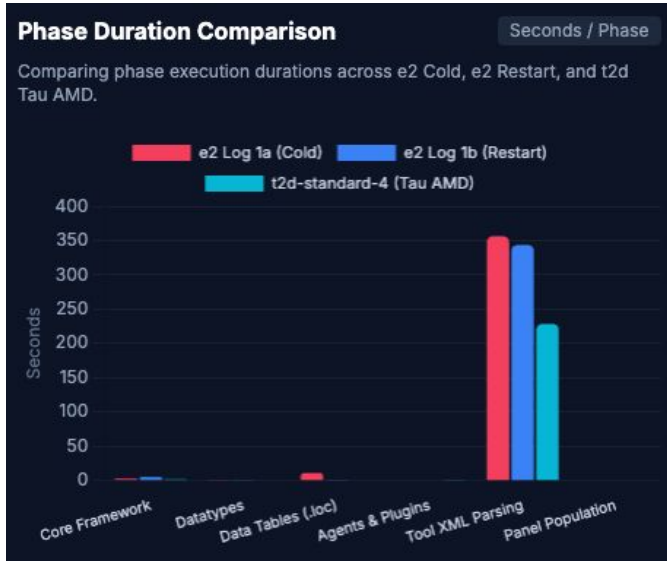
4. Helm Install galaxy-deps 47.43s

Deploying dependencies (PostgreSQL, Redis, RabbitMQ) into the cluster (4.7%).

Remaining 16 minor tasks account for less than 11.2% (111.9s combined).

More on
startup

Timings for helm install galaxy



Reading hundreds of files from CVMFS to build Galaxy's toolpanel is slow

[Galaxy 26.2 adds lazy toolbox loading](#), which should speed this up, hopefully to the target 5-6 mins

Compute cost: proportional to the workload

Idle-time costs are substantially lower, and compound

Workload costs tend to be less than what a comparably-sized GKE cluster could process in the same amount of time



Largely independent maintenance

Upgrades were a month+ long process involving 3 groups

Now, (mostly) independent

galaxy-k8s-boot

v1.0

v1.1

v2.0

anvil
branch

Machine image

Galaxy Helm

Galaxy



Terra / Leonardo



Current status

Spent the past 2 weeks working on the integration (it's a slow, iterative process)

Core tested features check out!

Data import & export to Terra still has some issues.

Next steps: push to Terra Dev to allow broader testing, and then Prod. Target release announcement by ACC.

Bringing AI to Galaxy on AnVIL

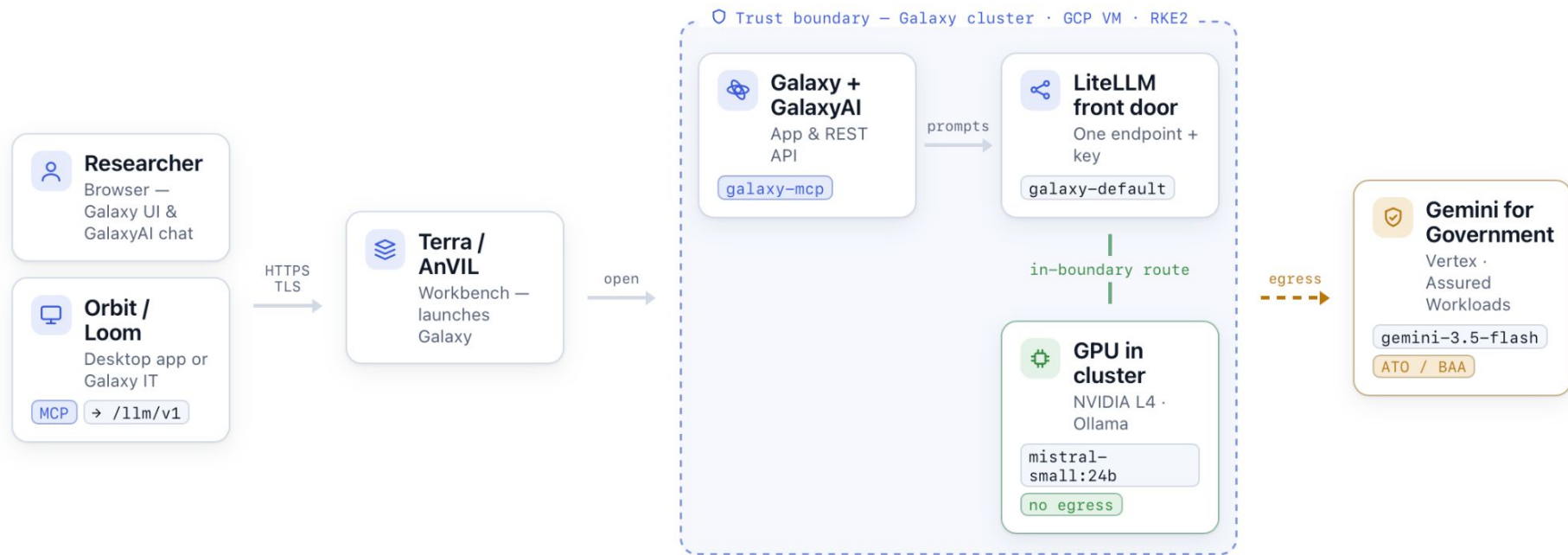
*GalaxyAI and Orbit/Loom
Two paths to conversational analysis*

The elephant in the room

Is it allowed and/or feasible

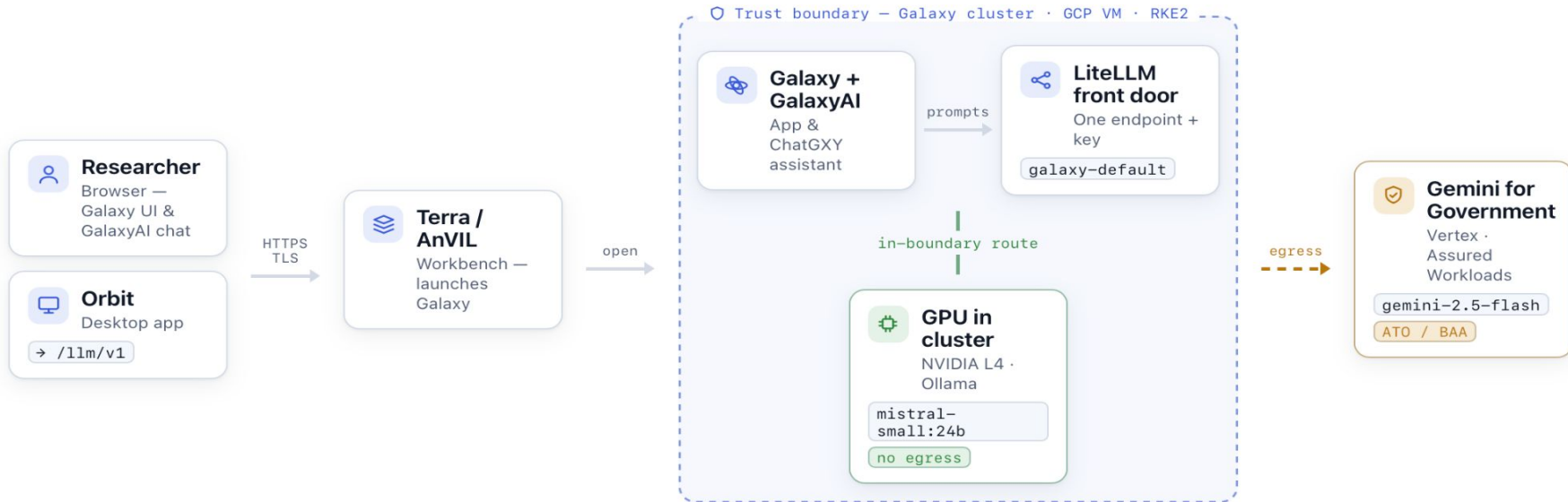
For the purposes of this presentation we will assume, yes it is allowed.

GalaxyAI Architecture



GalaxyAI — user access & LLM routing

galaxyai.useanvil.org · us-central1 · TLS



Trust boundary (cluster / VPC) Self-hosted — data stays in-boundary Egress — data leaves to a Google service

Two complementary approaches

Natural-language help to discover tools, design workflows, troubleshoot analyses, and make sense of results

INSIDE GALAXY

GalaxyAI

An AI assistant built into Galaxy itself, rendered in the web UI and natively aware of your tools, workflows, and histories.

ALONGSIDE GALAXY

Orbit / Loom

A standalone AI agent client that connects to a Galaxy instance – on the desktop or launched inside Galaxy as an Interactive Tool.

GalaxyAI

Server side, in the UI

- Part of the Galaxy application. No separate app to install

Natively Galaxy aware

- Agents understand the instance's tools, workflows, and histories directly.

Assists within Galaxy

- Find tools, design and repair workflows, and explain results in place.

GalaxyAI

One front door, swappable back ends



- LiteLLM in-cluster
Galaxy talks only to LiteLLM; the model behind it can change without touching Galaxy.
- No API key for Vertex (Google Gemini)
LiteLLM authenticates with the VM service account (ADC over the metadata server).
- Deployed as config
Turned on through *galaxy-k8s-boot*. Reproducible with the rest of the stack.

Orbit / Loom

Loom is the framework

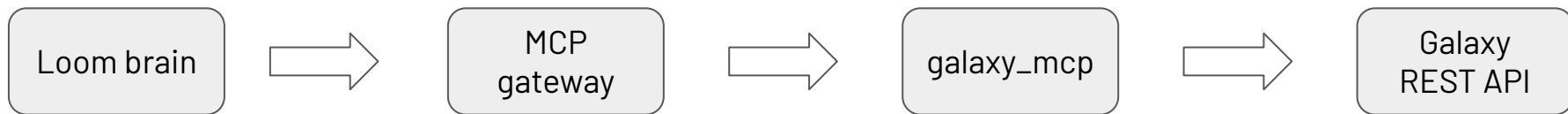
- An open agent runtime: a pi.dev based "brain," a web server, and MCP tool integration
- Communicates via Galaxy's REST API

Orbit is the client

- Runs standalone on the desktop
- Or as an Interactive Tool inside Galaxy

Orbit / Loom

How it reaches Galaxy



Speaks MCP

galaxy-mcp translates the agent's requests into Galaxy REST API calls.

Shared LLM path

The Loom *brain* points to the same LiteLLM front door

Acts as the user with the same permissions

GalaxyAI vs Orbit / Loom

Dimension	GalaxyAI	Orbit / Loom
Where it runs	Server side, inside Galaxy	External client
How it reaches Galaxy	Internal API	REST API
Authentication	Instance wide	Per user
Scope	Galaxy only	General purpose agent
Model backend	One shared instance	Its own config

They can share the same LLM backend, but GalaxyAI is a *capability of Galaxy* while Orbit is a *client that talks to Galaxy*.

Examples - Orbit / Loom

8

I have a human genome in the "Variant Test" history and I would like to perform a variant analysis to find indels. There is a reference genome in the history that you should use.

Understood! We will perform a variant analysis using the datasets in your "Variant Test" history, focusing on identifying indels (insertions and deletions). Here's what we need to do:

1. Identify Input Data:

- Locate the reference genome.
- Identify the sequencing data files.

2. Select Tools for Variant Analysis:

- Use `bwa` for aligning reads to the reference genome.
- Perform variant calling using tools like `samtools` and `bcftools`.

3. Run Workflow Steps:

- Align the sequencing data to the reference genome.
- Call variants from the aligned BAM file.

4. Filter for Indels:

- Use tools to filter out only indel variants (insertions and deletions).

5. Documentation:

- Keep detailed notes of each step, including parameters used and outputs generated.

Examples - GalaxyAI

 GalaxyAI

Hide Chats + New

Which tool should I use to trim adapters and low-quality bases from FASTQ reads?

 **Recommended Tools:**

- Trim Galore!** (ID: `toolshed.g2.bx.psu.edu/repos/bgruening/trim_galore/trim_galore/0.6.10+galaxy0`)
 - Quality and adapter trimmer of reads using cutadapt; automatically detects adapters, performs quality trimming, and produces QC reports.
- Trimmomatic** (ID: `toolshed.g2.bx.psu.edu/repos/pjbriggs/trimmomatic/trimmomatic/0.39+galaxy2`)
 - Flexible read trimming tool for Illumina NGS data; can remove adapters, trim low-quality bases, and filter reads.

Parameter Recommendations:

- Trim Galore!: {'adapter': 'auto (default) or specify adapter sequence with --adapter', 'quality': 'use --quality 20 to trim bases with Phred <20', 'length': 'use --length 30 to discard reads shorter than 30 bp after trimming'}
- Trimmomatic: {'adapters': 'provide adapter file with ILLUMINACLIP:adapters.fa:2:30:10', 'quality': 'use SLIDINGWINDOW:4:20 to cut when average quality drops below 20 over 4 bases', 'minlen': 'use MINLEN:30 to drop short reads'}

Live Demo

Questions